



Sawira Shakeel Raja

0000674882

BSCS-V

Assignment-II

Artificial Intelligence

Submitted To:

Mr. Touqeer Abbas

Date:

16th june-2026

Question 1: Neurons and Neural Networks

Part-a

a) Define the following terms in one or two lines each:

- Neuron (in a neural network)
- Weight
- Bias
- Activation function

Answer:

1. Neuron (in a Neural Network)

A neuron is the basic processing unit of a neural network, similar to a brain cell. It receives input values, performs mathematical calculations using weights and bias, and generates an output. Many neurons work together to learn patterns and make predictions.

2. Weight

A weight is a numerical value assigned to each input of a neuron that determines its importance. Larger weights have a greater influence on the neuron's output than smaller weights. During training, these weights are updated to improve the model's accuracy.

3. Bias

Bias is an additional value added to the weighted sum before applying the activation function. It allows the neuron to produce a meaningful output even when all input values are zero. Bias helps improve the flexibility and performance of the neural network.

4. Activation Function

An activation function is a mathematical function that converts the neuron's calculated value into the final output. It helps the neural network learn complex patterns by deciding whether a neuron should be activated. Common activation functions include **Sigmoid**, **ReLU**, and **Tanh**.

Part-b

A single neuron has two inputs. Calculate its output. Show every step.

[Given values:

Inputs: $x_1 = 1.0$, $x_2 = 0.5$

Weights: $w_1 = 0.6$, $w_2 = 0.4$

Bias: $b = 0.1$

Use the sigmoid activation: $f(z) = 1 / (1 + e^{-z})$

Steps to do:

- Step 1: Compute the weighted sum $z = (w_1 \times x_1) + (w_2 \times x_2) + b$
- Step 2: Apply the sigmoid function to get the output $y = f(z)$
- (Hint: $\text{sigmoid}(0.9) \approx 0.71$)

Answer:

Step 1: Compute the Weighted Sum (z)

Formula:

$$z = (w_1 \times x_1) + (w_2 \times x_2) + b$$

Substitute the given values:

$$z = (0.6 \times 1.0) + (0.4 \times 0.5) + 0.1$$

Calculate each term:

$$z = 0.6 + 0.2 + 0.1$$

$$z = 0.9$$

Therefore, the weighted sum is:

$$z = 0.9$$

Step 2: Apply the Sigmoid Function

Formula:

$$y=f(z)=1/1+e^{-z}$$

Substitute $z=0.9$

$$y=1/1+e^{-0.9}$$

Since,

$$e^{-0.9} \approx 0.4066$$

Now,

$$y=1/1+0.4066$$

$$y=1+0.4066 \quad y=1/1.4066$$

$$y \approx 0.7109$$

$$y \approx 0.71$$

Final Answer

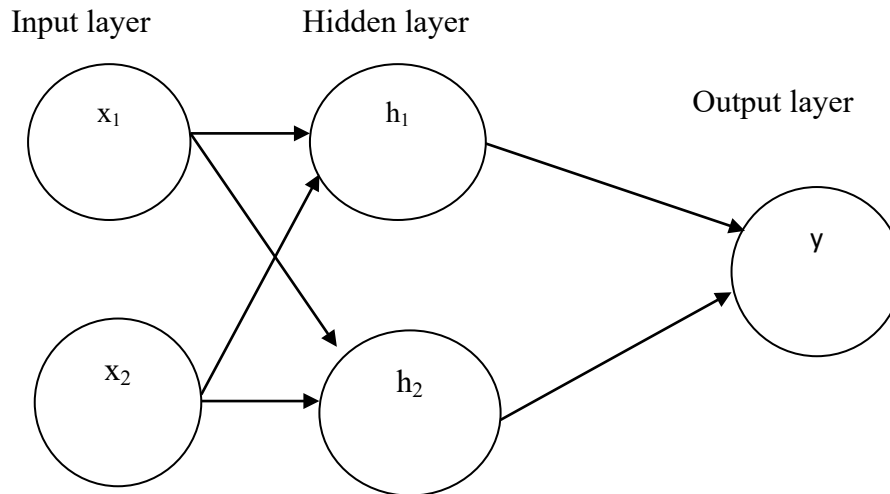
- **Weighted Sum (z) = 0.9**
- **Neuron Output (y) $\approx$ 0.71**

Therefore, the output of the neuron is: 0.71

Part-c

- Draw a simple neural network with an Input layer, one Hidden layer, and an Output layer. Label each layer.
- In one line: what is the difference between a forward pass and backpropagation?

1.Simple Neural Network Diagram:



Labels:

- **Input Layer:** Receives the input data (x_1, x_2).
- **Hidden Layer:** Processes the input using neurons.
- **Output Layer:** Produces the final prediction or output (y).

Difference between Forward Pass and Backpropagation:

Forward Pass is the process in which input data moves through the neural network to calculate the final output. It uses the current weights and bias to make a prediction.

Backpropagation works after the forward pass by calculating the error and updating the weights and bias to improve the network's accuracy during training.

Question 2: Natural Language Processing (NLP)

Part-a

Answer the following short questions:

1. What is NLP?

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, interpret, and generate human language. It helps machines process text and speech so they can interact with people naturally.

Example: Google Translate translates text from one language to another.

2. What is Syntax Analysis (Parsing)?

Syntax Analysis (Parsing) is the process of checking whether a sentence follows the correct grammatical structure.

Example: *"The cat is sleeping."* is a grammatically correct sentence.

3. What is Morphology in NLP?

Morphology is the study of the structure and formation of words in a language.

Example: The word **"playing"** consists of the root word **"play"** and the suffix **"-ing"**.

4. Name any two applications of NLP from daily life.

- **Google Translate** (Language Translation)
- **Siri** (Voice Assistant)

Part-b

Explain Tokenization, Stemming, and Lemmatization.

Technique	What it does	Input words	Your answer
Tokenization	Splits text into words	"I love AI"	I, love, AI
Stemming	Removes suffixes to find the root	playing	play
Lemmatization	Finds the dictionary root	better	good

Coding implementation:

Step 1: Install Libraries

Explanation

In this step, the required Python libraries are imported. **NLTK** is used for text preprocessing tasks such as stop-word removal, stemming, and lemmatization. These libraries provide the tools needed to process natural language text.

Code and output:

```
[ ] ▶ # Install required libraries (Run only once in Google Colab)
|pip -q install nltk

import nltk
import re

# Download required datasets
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

▼ ... [nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
```

Step 2: Input Text

Explanation

The original sentence is given as input to the NLP pipeline. This text will pass through different processing steps so that the computer can understand and analyze it.

Code and output:

```
▶ text = "I love Artificial Intelligence because it is making our lives better by playing an important role."

print("="*60)
print("STEP 1 : INPUT TEXT")
print("="*60)
print(text)

... =====
STEP 1 : INPUT TEXT
=====
I love Artificial Intelligence because it is making our lives better by playing an important role.
```

Step 3: Text Preprocessing

Explanation

The input text is converted into lowercase and unnecessary characters or punctuation marks are removed. This makes the text clean and ensures that all words are processed consistently.

Code and output:

```

▶ text = text.lower()

text = re.sub(r'^a-zA-Z\s|', '', text)

print("\n" + "="*60)
print("STEP 2 : TEXT PREPROCESSING")
print("="*60)
print(text)

```

```

...
=====
STEP 2 : TEXT PREPROCESSING
=====
i love artificial intelligence because it is making our lives better by playing an important role

```

Step 4: Tokenization

Explanation

Tokenization splits the sentence into individual words called **tokens**. These tokens become the basic units that the NLP system processes in the following steps.

Code and output:

```

▶ tokens = text.split()

print("\n" + "="*60)
print("STEP 3 : TOKENIZATION")
print("="*60)
print(tokens)

```

```

...
=====
STEP 3 : TOKENIZATION
=====
['i', 'love', 'artificial', 'intelligence', 'because', 'it', 'is', 'making', 'our', 'lives', 'better', 'by', 'playing', 'an', 'important', '']

```

Step 5: Stop-word Removal

Explanation

Common words such as **is**, **the**, **an**, **our**, **by** are removed because they usually do not add important meaning to the sentence. This helps the model focus on the meaningful words.

Code and output:


```

stop_words = set(stopwords.words('english'))

filtered_tokens = []

for word in tokens:

    if word not in stop_words:

        filtered_tokens.append(word)

print("\n" + "="*60)
print("STEP 4 : STOP WORD REMOVAL")
print("="*60)

print(filtered_tokens)

```

```

=====
STEP 4 : STOP WORD REMOVAL
=====
['love', 'artificial', 'intelligence', 'making', 'lives', 'better', 'playing', 'important', 'role']

```

Step 6: Stemming

Explanation

Stemming reduces words to their root form by removing prefixes or suffixes. For example, **playing** becomes **play**. This helps group similar words together, although the result may not always be a valid dictionary word.

Code and output:

```

▶ stemmer = PorterStemmer()

stemmed_words = []

for word in filtered_tokens:

    stemmed_words.append(stemmer.stem(word))

print("\n" + "="*60)
print("STEP 5 : STEMMING")
print("="*60)

print(stemmed_words)

...
=====
STEP 5 : STEMMING
=====
['love', 'artifici', 'intellig', 'make', 'live', 'better', 'play', 'import', 'role']

```

Step 7: Lemmatization

Explanation

Lemmatization converts words into their correct dictionary (base) form. Unlike stemming, it uses vocabulary rules to produce meaningful words. For example, **lives** becomes **life**.

Code and output:

```

▶ lemmatizer = WordNetLemmatizer()

lemmatized_words = []

for word in filtered_tokens:

    lemmatized_words.append(lemmatizer.lemmatize(word))

print("\n" + "="*60)
print("STEP 6 : LEMMATIZATION")
print("="*60)

print(lemmatized_words)

...

=====
STEP 6 : LEMMATIZATION
=====
['love', 'artificial', 'intelligence', 'making', 'life', 'better', 'playing', 'important', 'role']
```

Step 8: Install Sentence Transformer

Explanation

The **Sentence Transformers** library is installed to use a pre-trained AI model. This model converts complete sentences into numerical vectors called **embeddings**, which capture the semantic meaning of the text.

Code and output:

```

▶ # Install Sentence Transformers
!pip -q install sentence-transformers

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
```

Step 9: Load AI Model

Explanation

The pre-trained **all-MiniLM-L6-v2** model is loaded into memory. This model has already learned language patterns from a large amount of text and can understand the meaning of sentences without additional training.

Code and output:

```
print("="*60)
print("STEP 7 : LOADING AI MODEL")
print("="*60)

model = SentenceTransformer('all-MiniLM-L6-v2')

print("AI Model Loaded Successfully!")
```

```
=====
STEP 7 : LOADING AI MODEL
=====
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster download.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable
modules.json: 100% ██████████ 349/349 [00:00<00:00, 6.97kB/s]

config_sentence_transformers.json: 100% ██████████ 116/116 [00:00<00:00, 1.34kB/s]
README.md: 100% ██████████ 10.5k/10.5k [00:00<00:00, 159kB/s]
sentence_bert_config.json: 100% ██████████ 53.0/53.0 [00:00<00:00, 728B/s]
config.json: 100% ██████████ 612/612 [00:00<00:00, 11.5kB/s]
model.safetensors: reconstructing file: 100% ██████████ 90.9MB / 90.9MB, 8.75MB/s
model.safetensors: downloading bytes: ██████████ 85.0MB, 7.93MB/s

model.safetensors: downloading bytes: ██████████ 85.0MB, 7.93MB/s
Loading weights: 100% ██████████ 103/103 [00:00<00:00, 2252.94it/s]
tokenizer_config.json: 100% ██████████ 350/350 [00:00<00:00, 8.72kB/s]
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 3.48MB/s]
tokenizer.json: 100% ██████████ 466k/466k [00:00<00:00, 9.63MB/s]
special_tokens_map.json: 100% ██████████ 112/112 [00:00<00:00, 9.31kB/s]
config.json: 100% ██████████ 190/190 [00:00<00:00, 14.7kB/s]
AI Model Loaded Successfully!
```

Step 10: Generate Sentence Embeddings

Explanation

The AI model converts each input sentence into a **384-dimensional embedding vector**. These vectors represent the meaning of the sentences in numerical form, allowing the computer to compare their meanings.

Code and output:

```
▶ sentence1 = "Artificial Intelligence makes our lives easier."
sentence2 = "AI helps people perform tasks more efficiently."

print("\n" + "="*60)
print("STEP 8 : INPUT SENTENCES")
print("="*60)

print("Sentence 1:", sentence1)
print("Sentence 2:", sentence2)

embedding1 = model.encode(sentence1)
embedding2 = model.encode(sentence2)

print("\nEmbedding 1 Shape:", embedding1.shape)
print("Embedding 2 Shape:", embedding2.shape)

print("\nFirst 10 Values of Embedding 1:")
print(embedding1[:10])

print("\nFirst 10 Values of Embedding 2:")
print(embedding2[:10])

=====
STEP 8 : INPUT SENTENCES
=====
Sentence 1: Artificial Intelligence makes our lives easier.
Sentence 2: AI helps people perform tasks more efficiently.

Embedding 1 Shape: (384,)
Embedding 2 Shape: (384,)

First 10 Values of Embedding 1:
[-0.00202122 -0.0024892  0.09226122  0.01044159  0.05502397  0.00014711
  0.02315046  0.03698228  0.06463546  0.00836258]

First 10 Values of Embedding 2:
[-0.01818246 -0.00585512  0.02323141 -0.00113682  0.00479347  0.03053193
  0.05216236  0.00987989  0.0354337  0.01952501]
```

Step 11: Similarity Calculation

Explanation

Cosine Similarity is used to measure how similar the two sentence embeddings are. A value closer to **1** means the sentences have very similar meanings, while a value closer to **0** means they are different.

Code and output:

```

▶ print("\n" + "="*60)
  print("STEP 9 : COSINE SIMILARITY")
  print("="*60)

  similarity = cosine_similarity(
      [embedding1],
      [embedding2]
  )

  print("Similarity Score:")
  print(similarity)
...
=====
STEP 9 : COSINE SIMILARITY
=====
Similarity Score:
[[0.68676376]]

```

Step 12: AI Model Inference

Explanation

The AI model analyzes the similarity score and classifies the relationship between the two sentences. Based on predefined thresholds, it predicts whether the sentences are **Highly Similar**, **Moderately Similar**, or **Different**.

Code and output:

```

▶ print("\n" + "="*60)
  print("STEP 10 : AI MODEL INFERENCE")
  print("="*60)

  score = similarity[0][0]

  if score > 0.80:
      prediction = "Highly Similar"

  elif score > 0.60:
      prediction = "Moderately Similar"

  else:
      prediction = "Different Meaning"

  print("Prediction:")
  print(prediction)
...
=====
STEP 10 : AI MODEL INFERENCE
=====
Prediction:
Moderately Similar

```

Step 13: Final Prediction

Explanation

The final output displays the embedding size, similarity score, and the predicted relationship between the sentences. In this implementation, the similarity score is **0.6867**, so the model predicts that the sentences are **Moderately Similar**.

Code and output:

```
print("\n" + "="*60)
print("FINAL NLP PIPELINE")
print("="*60)

print("Original Sentence 1:")
print(sentence1)

print("\nOriginal Sentence 2:")
print(sentence2)

print("\nEmbedding Size:")
print(len(embedding1))

print("\nSimilarity Score:")
print(score)

print("\nFinal Prediction:")
print(prediction)
```

...

```
=====
FINAL NLP PIPELINE
=====
Original Sentence 1:
Artificial Intelligence makes our lives easier.

Original Sentence 2:
AI helps people perform tasks more efficiently.

Embedding Size:
384

Similarity Score:
0.68676376

Final Prediction:
Moderately Similar
```

Part-c

Write 3–4 lines on recent advances in NLP (for example: ChatGPT, Transformers, voice assistants, or machine translation).

Answer:

Recent Advances in NLP:

Recent advances in NLP have made computers much better at understanding human language. Transformer-based models such as ChatGPT can answer questions, summarize text, and generate content. Voice assistants like Siri and Google Assistant understand spoken commands more accurately. Machine translation tools have also improved and can translate languages with higher accuracy.